

GESTIÓN DE APLICACIONES MODERNAS EN LA NUBE

R. A. Ramírez Gómez
7690-22-12700 Universidad Mariano Gálvez
Seminario de Tecnologías de Información
rramirezg18@miumg.edu.gt

Resumen

La presente investigación es acerca de los componentes que intervienen en la gestión de una aplicación moderna en la nube. Se investigó sobre la arquitectura de microservicios, la orquestación de contenedores mediante Kubernetes, la metodología de las doce factores para aplicaciones nativas de la nube y el estándar OAuth 2.0 para la autorización de servicios, apoyándose en documentación oficial y estándares reconocidos. Se explica cómo estos elementos se complementan entre sí: los microservicios dividen la aplicación, los contenedores la empaquetan, la orquestación los administra, la metodología de doce factores guía su construcción y OAuth 2.0 protege el acceso a los servicios. Se concluye que la gestión de una aplicación moderna en la nube no depende de una sola tecnología, sino de la combinación coordinada de estos componentes.

Palabras clave: microservicios, Kubernetes, orquestación de contenedores, OAuth 2.0, aplicaciones nativas de la nube

Desarrollo del tema

El desarrollo de software ha evolucionado hacia aplicaciones distribuidas que se ejecutan en la nube. En este modelo, la aplicación suele dividirse en microservicios, que son servicios pequeños e independientes que se comunican entre sí mediante interfaces de programación de aplicaciones (API), en lugar de construirse como un único bloque monolítico (Fowler y Lewis, 2014). Sobre esta base, existe un conjunto de tecnologías y metodologías que trabajan de forma conjunta para gestionar dichas aplicaciones. A continuación se explican los principales componentes que intervienen en esta gestión.

Orquestación de servidores

Cuando una aplicación se divide en muchos servicios, cada uno se empaqueta en un contenedor, que es un paquete ligero y autónomo que incluye todo lo necesario para ejecutar el servicio, como el código, las librerías y las dependencias. La herramienta más conocida para crear y ejecutar contenedores es Docker, que popularizó esta tecnología a partir de 2013, y a diferencia de las máquinas virtuales, los contenedores comparten el mismo núcleo del sistema operativo del equipo que los aloja, por lo que consumen menos recursos y son más rápidos.

La orquestación es el proceso de automatizar el despliegue, la gestión, el escalado y la conexión de estos contenedores y de los servidores donde se ejecutan, a lo largo de su ciclo de vida (Kubernetes, 2024). Cuando una aplicación maneja cientos o miles de contenedores distribuidos en varios servidores, administrarlos de forma manual se vuelve ineficiente y propenso a errores, por lo que se necesita una herramienta que coordine automáticamente en qué servidor se ejecuta cada contenedor, que los reinicie si fallan y que distribuya la carga de trabajo de forma equilibrada.

Sin la orquestación, un equipo de desarrollo tendría que decidir manualmente en qué servidor colocar cada contenedor, vigilar de forma constante que todos sigan funcionando y reaccionar a mano cuando alguno falla o cuando aumenta la demanda, lo cual resulta inviable a gran escala. Además de Kubernetes, existen otras herramientas de orquestación como Docker Swarm y Apache Mesos, aunque Kubernetes se ha consolidado como la opción más utilizada en la industria por su robustez y por la amplia comunidad que la respalda.

Kubernetes

La herramienta que se ha convertido en el estándar de la industria para la orquestación es Kubernetes, un motor de código abierto que automatiza el despliegue, el escalado y la gestión de aplicaciones en contenedores, mantenido por la Cloud Native Computing Foundation (Kubernetes, 2024). Kubernetes funciona mediante un modelo declarativo, en el que el desarrollador describe el estado deseado del sistema y la herramienta se encarga de mantener la realidad ajustada a ese estado de forma automática.

Su estructura se organiza en clústeres, que representan el conjunto completo del sistema; nodos, que son las máquinas o servidores individuales que realizan el trabajo; y pods, que son las unidades más pequeñas que se pueden desplegar y que contienen uno o más contenedores de la aplicación (Kubernetes, 2024). Gracias a esta organización, Kubernetes distribuye la carga entre los distintos nodos, reinicia automáticamente los contenedores que fallan, escala la cantidad de réplicas de un servicio según la demanda y facilita el descubrimiento y la comunicación entre los servicios, reduciendo de forma significativa el trabajo manual que implicaría administrar todo esto por cuenta propia.

Entre las ventajas que ofrece Kubernetes se encuentran la alta disponibilidad, ya que si un servidor deja de funcionar la herramienta redistribuye los contenedores hacia otros nodos disponibles; la escalabilidad automática, que permite ajustar los recursos según la carga real del sistema; y la portabilidad, dado que una misma configuración puede ejecutarse en distintos proveedores de nube o en servidores propios. No obstante, su uso también implica una curva de aprendizaje considerable y una mayor complejidad de configuración, por lo que se recomienda principalmente para aplicaciones que realmente requieren gestionar una gran cantidad de contenedores.

Autorización con OAuth 2.0

Cuando una aplicación se compone de múltiples servicios que se comunican entre sí y con aplicaciones externas, la seguridad en el acceso se vuelve fundamental. OAuth 2.0 es el estándar de la industria para la autorización y, según la especificación oficial, permite que una aplicación de terceros obtenga acceso limitado a un servicio HTTP sin necesidad de compartir la contraseña del usuario, utilizando en su lugar un token de acceso con un alcance y un tiempo de vida determinados (IETF, 2012).

OpenWebinars (2020) describe los cuatro roles que participan en este proceso: el dueño del recurso, que es el usuario que autoriza el acceso; el cliente, que es la aplicación que solicita el acceso; el servidor de recursos, que aloja la información protegida; y el servidor de autorización, que emite los tokens una vez que el usuario da su consentimiento. El conjunto de permisos que se le concede a la aplicación se conoce como alcance, lo que permite que el usuario decida exactamente qué puede hacer la aplicación en su nombre, por ejemplo, otorgar únicamente acceso de lectura. Este mecanismo es el que permite iniciar sesión en una aplicación usando una cuenta de Google sin entregarle la contraseña, y es utilizado hoy por grandes compañías como Google, Facebook, Microsoft y GitHub.

OAuth 2.0 define además distintos flujos de autorización que se adaptan a cada situación, como el flujo de código de autorización, que es el más común en aplicaciones web; el flujo de credenciales de cliente, utilizado en la comunicación directa entre servicios sin intervención de un usuario; y el flujo con clave de prueba, recomendado para aplicaciones móviles por ofrecer mayor seguridad. Contar con distintos flujos permite que este estándar se aplique de forma adecuada tanto en aplicaciones web y móviles como en la comunicación interna entre microservicios, lo cual lo convierte en una pieza clave para la seguridad de las aplicaciones modernas en la nube.

Es importante aclarar que OAuth 2.0 es un estándar de autorización y no de autenticación, es decir, se encarga de definir qué puede hacer una aplicación con los recursos de un usuario, y no de verificar la identidad de ese usuario. Para la autenticación existe una capa construida sobre OAuth 2.0 llamada OpenID Connect, que se utiliza cuando además de otorgar permisos se necesita confirmar quién es el usuario. Distinguir entre ambos conceptos es fundamental para implementar correctamente la seguridad en una aplicación distribuida.

Implementación de servicios en la nube (12-factor)

Para que una aplicación funcione correctamente al implementarse en la nube, no basta con dividirla en servicios y orquestrarla, sino que también debe construirse siguiendo ciertos principios. La metodología de las doce factores, creada por ingenieros de Heroku, es un conjunto de principios para construir aplicaciones de software como servicio que sean portables y escalables en la nube (Wiggins, 2017). Según Wiggins (2017), busca que las aplicaciones utilicen formatos declarativos para automatizar su configuración, ofrezcan máxima portabilidad entre entornos, minimicen la divergencia entre desarrollo y producción, y puedan escalar sin cambios significativos en su arquitectura.

Entre sus doce principios se encuentran mantener una única base de código versionada, declarar explícitamente las dependencias, almacenar la configuración en el entorno y no en el código, tratar los servicios de respaldo como recursos conectables, separar de forma estricta las etapas de construcción y ejecución, y tratar los registros como flujos de eventos. Estos principios son especialmente compatibles con la arquitectura de microservicios y con el despliegue en contenedores, ya que todos comparten el objetivo de lograr aplicaciones portables, escalables y fáciles de desplegar de forma automatizada.

Dos de estos principios resultan especialmente relevantes al implementar servicios en la nube. El primero es almacenar la configuración en el entorno y no en el código, lo cual significa que datos como las contraseñas de las bases de datos o las claves de acceso no deben escribirse directamente en el código fuente, sino inyectarse desde el entorno de ejecución, permitiendo que la misma aplicación se despliegue en distintos entornos sin modificar su código. El segundo es tratar los registros como flujos de eventos, es decir, que la aplicación no debe encargarse de gestionar sus propios archivos de registro, sino emitirlos como un flujo continuo que un sistema externo se encargue de recolectar y analizar, lo cual facilita el monitoreo de aplicaciones distribuidas en la nube.

Integración de los componentes

En conjunto, la orquestación administra los contenedores en los servidores, Kubernetes automatiza esa gestión, OAuth 2.0 protege el acceso y la metodología de las doce factores guía la construcción, de modo que cada componente resuelve una parte de la gestión de la aplicación en la nube.

Estos componentes suelen integrarse dentro de un flujo de trabajo automatizado conocido como integración y entrega continua (CI/CD, por sus siglas en inglés). En este flujo, cuando un desarrollador

realiza un cambio en el código de un microservicio, el sistema construye automáticamente una nueva imagen de contenedor siguiendo los principios de las doce factores, la despliega en el clúster de Kubernetes y aplica las políticas de seguridad correspondientes, mientras que OAuth 2.0 continúa protegiendo el acceso a los servicios. De esta manera, la combinación de estas tecnologías no solo permite gestionar la aplicación una vez desplegada, sino también automatizar todo el ciclo desde el desarrollo hasta la producción, lo cual reduce los errores manuales y acelera la entrega de nuevas versiones.

Observaciones y comentarios

Considero que uno de los mayores retos de este modelo no es entender cada componente por separado, sino comprender cómo se relacionan entre sí, ya que es fácil quedarse solo con los nombres de las tecnologías sin ver el panorama completo. También pude notar que muchas veces se adoptan estas herramientas por moda, sin evaluar si el proyecto realmente justifica la complejidad que introducen, cuando en aplicaciones pequeñas un enfoque más simple podría ser más conveniente.

Conclusiones

1. Considero que el mayor valor de estas tecnologías no está en usarlas por separado, sino en entender cómo se combinan, ya que es esa integración la que realmente permite gestionar una aplicación moderna en la nube.
2. En mi opinión las herramientas como Kubernetes son muy poderosas, pero pienso que muchas veces se adoptan por moda sin evaluar si el proyecto justifica la complejidad que agregan.
3. La seguridad con OAuth 2.0 es tan importante como la propia gestión de los servicios, porque de nada sirve una aplicación bien orquestada si el acceso a sus datos no está bien protegido.

Referencias

- Fowler, M., & Lewis, J. (2014). *Microservices: A definition of this new architectural term*. Thoughtworks / martinoflower.com. <https://martinfowler.com/articles/microservices.html>
- IETF. (2012). *RFC 6749: The OAuth 2.0 authorization framework* (D. Hardt, Ed.). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6749.html>
- Kubernetes. (2024). *Overview / Kubernetes documentation*. Cloud Native Computing Foundation. <https://kubernetes.io/docs/home/>
- OpenWebinars. (2020). *Qué es OAuth 2, ventajas, roles y flujo*. <https://openwebinars.net/blog/que-es-oauth2/>
- Wiggins, A. (2017). *The twelve-factor app*. <https://12factor.net/>

Código fuente: <https://github.com/rramirezg18/foros/tree/main/foro5>